Two Planes

the data/control distinction, and the three things it forces

Robin Langer Claude

21 September 2026

Abstract

A container morphism is a pair (u, f) of opposite variance: u dispatches on shapes, f routes positions, and f runs backwards. Shapes form a bare set and positions carry structure, so the two components live in different categories. This note argues that the asymmetry is not incidental notation but the load-bearing wall of the formalism, and that one prohibition follows from it — *a container morphism may not look at the payload* — from which three otherwise unrelated results follow.

First, redundancy is available on one plane only: coding needs an addition, and shapes have none, so the control plane must be protected by replication rather than by codes (`error-correcting.pdf`). Second, a federation carries two independent obstructions, not one — well-foundedness in u and a class in H^1 in f — which resolves a question left open in `sheaf-federation.pdf` and is verified on running servers. Third, the flag container's Hecke algebra is an algebra of *dispatch*, which is why Schubert polynomials, needing graded positions, do not appear (`schubert.pdf`).

We then record what cannot be repaired: linearising the shapes collapses the two planes into one and forgets the shape set entirely. And we state the limit honestly — real systems need data to influence control, so the formalism describes not a whole system but the fragment that can be checked statically. Its value is that it tells you exactly where that fragment ends.

1 The distinction

Definition 1.1 (the two planes)**.** A container is $S \triangleleft P$: a set S of *shapes*, and for each s an object Ps of *positions*. A morphism $(S \triangleleft P) \to (T \triangleleft Q)$ is a pair

$$u : S \longrightarrow T, \qquad f_s : Q(u\,s) \longrightarrow Ps.$$

We call u the *control plane* and f the *data plane*.

Three features, and each does work below.

(i) **Opposite variance.** Control flows forward, data flows back.

(ii) **Different categories.** S lives in **Set**; Ps lives in **Vect**, in $\mathbb{F}_q$-spaces, or in whatever the base is.

(iii) **One-way dependence.** f may depend on s. u may not depend on the data.

Principle 1.2 (the prohibition)**.** *A container morphism may not look at the payload. Equivalently: there is no data → control flow.*

Everything in this note is a consequence of Principle 1.2 or of the fact that (ii) is what sustains it.

2 First instance: redundancy lives on one plane

Theorem 2.1 (`error-correcting.pdf`)**.** *Linear redundancy is available for positions and unavailable for shapes.*

Reason. Redundancy means forming linear combinations, which requires an addition on the values protected. Positions span a vector space and have one. Shapes form a bare set: there is no $s_1 + s_2$. □

Principle 2.2 (two planes, two mechanisms)**.** *Protect positions with codes. Protect shapes with replication, acknowledgement or consensus.*

The check that this is more than bookkeeping is that practice already obeys it and did not know why. Packet headers are protected by checksum-and-discard, never by coding across headers — a corrupted header means the packet is dropped, not repaired. Storage systems replicate metadata and erasure-code data. A system that erasure-codes its payload and leaves its metadata on a single unreplicated path has not been made reliable; it has moved its single point of failure into the header. The formalism says this was not a design choice.

3 Second instance: a federation has two obstructions

This is the new claim of the present note, and it reinterprets a result obtained independently in `trace-and-class.pdf`.

A Sheaf Theory of Federation asked whether the trace obstruction on $\triangleleft$ and the class in H^1 are the same obstruction, offering two answers: they are the same, or one of the two models is wrong. Both are too tight.

Proposition 3.1. *In the ring of container morphisms, well-foundedness is a property of u and the cohomology class is a property of f. Since the two components of a morphism are chosen independently, the two obstructions are independent.*

The evidence is that the four handlers of the reference implementation all apply the transition function ε in the same position — to the reply, on the way back, which is the only position descent allows — and differ only in what stops them: fuel, fuel recorded in the response type, a visited list, or nothing at all. What distinguishes the diverging handler from the terminating ones is therefore a property of the forward map's control flow, where no ε occurs. Writing the recursion out, $\mathrm{Orient}(i) = \varepsilon_i \cdot \mathrm{Orient}(i+1)$, the argument never mentions ε.

class	repair	H^1	terminates	canonical	witness
cylinder	none	0	no	—	abandoned at 8s, 757 hops
cylinder	cut	0	yes	yes	all three cuts agree
mobius	none	$\neq 0$	no	—	abandoned at 6s, 660 hops
mobius	cut	$\neq 0$	yes	no	the three cuts disagree

All four cells populate, on three live servers. The load-bearing cell is the first: on the cylinder the twisted sheaf has two global sections, nothing whatever obstructs descent, and the unbounded walk still does not return. A vanishing class did not buy termination, because termination was never what the class was about.

Corollary 3.2. *The two planes obstruct different properties. Well-foundedness governs existence — is there an answer at all. The class governs canonicity — is the answer independent of the choice made to obtain it. Cutting an edge trades divergence for choice, and H^1 is exactly the price of the choice.*

Checked by enumeration on all eight elements of $(\mathbb{Z}/2)^3$, together with the gauge invariance — all 24 gauge transformations move the cocycle and never move the verdict — which is what makes the statement one about H^1 rather than about a chosen representative.

Remark 3.3. The sheaf model sees only the data plane, because H^1 is all it offers. The operational model sees only the control plane, because divergence is all it offers. Neither is wrong; each is complete on its own plane. That is why the question as posed had no good answer.

4 Third instance: the flag container's algebra is an algebra of dispatch

The flag container is $\sum_{w \in S_n} X^{\ell(w)}$: shapes are the permutations, positions over shape w are the Bruhat cell $\mathbb{F}_q^{\ell(w)}$. So the Weyl group, the Bruhat order and the Hecke algebra all live on the *control* plane, and the cells are the data.

`schubert.pdf` shows that the fibration forgetting the i-th subspace gives S_i with $S_i^2 = (q+1)S_i$, hence $T_i = S_i - I$ satisfying the Hecke quadratic, with the braid relations holding and no nilpotent attached to any single index. Read through the present distinction, that result says: *the algebra the container produces is the algebra of how it dispatches.*

Corollary 4.1. *Schubert polynomials are not reachable because they are a data-plane refinement. $\mathfrak{S}_w(x_1, \ldots, x_n)$ records the torus weights on positions; counting $\mathbb{F}_q$-points records only how many positions there are. The one-variable shadow of $\mathfrak{S}_w$ is $x^{\ell(w)}$, and the whole container decategorifies to the Poincaré polynomial $\sum_w x^{\ell(w)}$, which at $x = q$ is $[n]_q!$.*

Question 4.2. Grade the positions by the torus action, so that a position carries a weight and not merely a dimension. Does the multigraded flag container have $\mathfrak{S}_w(x_1, \ldots, x_n)$ as its shape polynomial?

5 Why the distinction cannot be repaired away

It is tempting to regard the set-ness of shapes as a limitation to be generalised away by making both planes linear. `q-container.pdf` Theorem B says what happens.

Theorem 5.1 (Theorem B, restated for this purpose)**.** *If the extension is taken to be $\bigoplus_s \mathrm{Hom}(Ps, V)$ rather than $\sum_s \mathrm{Hom}(Ps, V)$, the natural transformations collapse to $\mathrm{Hom}\big(\bigoplus_t Qt, \bigoplus_s Ps\big)$, which forgets the shape set entirely.*

Corollary 5.2. *The distinction survives precisely because the two planes live in different categories. Collapse them into one and one does not obtain a more general theory; one obtains a single plane, and the control plane is the one that disappears.*

So (ii) is the load-bearing wall, and Theorem 2.1 is not an accident of presentation: shapes have no addition because shapes are in **Set**, and they are in **Set** because that is the only thing keeping two planes from becoming one.

6 The agentic reading

Reading 6.1 (stated as a reading, not a theorem). Prompt injection is data becoming control: a payload read by the dispatcher. Principle 1.2 says a container morphism cannot do this, since u is a function of the shape alone. The vulnerability class is therefore not mitigated within the formalism; it is *unexpressible* in it.

Two qualifications, because the claim is easy to overstate. Unexpressible in the fragment is not the same as absent from the system: a real agent does read its inputs and branch, and that part of it is outside the formalism by §7. What the discipline buys is that the unsafe flow must be written somewhere explicit, rather than arising by accident anywhere.

And it explains the status of labels carried through a system. A trust label that is transported unchanged by f and never consulted by u is a *passenger*: it needs no type discipline of its own, precisely because it never reaches the control plane.

7 What is outside, and the limit

operation	morphism?	why
encoding, syndrome extraction $s = Hy$	yes	a matrix
erasure decoding, pattern known	yes	pattern is metadata, recovery a matrix
error correction, position unknown	**no**	must infer from the payload
retransmission, rate adaptation	**no**	reads state and branches

The line falls in exactly one place, every time: an operation is outside when data influences control. Note the split *inside* decoding — computing a syndrome is linear and therefore a morphism, while mapping a syndrome to an error vector is a search and therefore not. The formalism separates the algebra of decoding from the search.

Remark 7.1 (the honest limit). Real systems need data → control flow: retransmission, rate adaptation, error correction, any adaptive agent. The formalism therefore does not describe a whole system. It describes the fragment that can be checked statically, and its value is that it tells you exactly where that fragment ends. As `error-correcting.pdf` says plainly, it yields no new codes; what it yields is recoverability as a property of declared interfaces, computable by multiplying matrices, rather than established per-code and per-deployment.

Principle 7.2 (what to do with a boundary)**.** *When a system needs data to influence control, that is not a failure to formalise. It is a boundary one should be able to point at. Containers make you point at it.*

8 Status

#	claim	source	verdict
1	redundancy unavailable for shapes	`error-correcting`	proved
2	the two federation obstructions are independent	`trace-and-class`	**confirmed**, live + 8/8
3	flag container's algebra is Hecke, not nil-Coxeter	`schubert`	proved + 5/5
4	linearising shapes collapses the planes	`q-container` Thm B	proved
5	Prop. 3.1: (1)–(4) are one distinction	this note	synthesis, new
6	injection is unexpressible in the fragment	this note	reading, not proved

Companions. `q-container.pdf`, `error-correcting.pdf`, `schubert.pdf` in this directory; `trace-and-class.pdf` and `independence.ts` in `gold/federation`.